

第12章 高级数据结构

建议学时:4-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解红黑树五条性质与插入修正,知道它是 2-3-4 树的二叉表示
- 掌握 Treap 与跳表的随机化平衡思想,能完整实现插入/删除/查找
- 掌握 k-d 树最近邻剪枝查询,理解配对堆摊还 $O(1)$ 合并
- 学会 Python 的"有序字典"替代惯例:list+bisect、dict+sorted、sortedcontainers,并实测 10^4 级跳表字典

💡 从 C 到 Python:本章主题如何迁移

前 11 章最大的缺口是"平衡":BST 遇有序输入退化成链表,AVL 实现繁琐。本章五个结构给出三种哲学:红黑树用确定不变式强制平衡;Treap 与跳表把平衡交给随机性(期望 $O(\log n)$);配对堆靠摊还分析兜底。

C 里做"有序字典"要手写几百行平衡树,一个修正分支写错就悄悄失衡。Python 没有内建平衡树,但有两条轻量惯例: `list + bisect` 二分(小数据)、`dict + sorted` (批处理),大数据再上第三方 `sortedcontainers` ——这正是本章"选型"的实用部分。

跳表与 Treap 的价值在"实现简单":代码量接近链表,却拿到期望 $O(\log n)$ ——用"概率上几乎不可能坏"换"写起来几乎不可能错",Redis 的 zset 底层就是跳表。Python 实现它们比 C 更短:指针变成列表索引, `new / delete` 变成对象引用与垃圾回收。

📦 前置知识自查

数学前置:概率期望直觉(几何分布)、对数运算、摊还分析概念(第11章)。

C 语言前置:`struct` 与指针、链表结点增删(第3章)、函数指针、`malloc / free` 对照 Python 的对象与引用计数。

依赖的前面章节:第4章树(BST、旋转、AVL)、第5章散列、第6章堆、第11章摊还分析。

🚀 本章学习路线

1. **第一阶段(2 小时):**读 §12.1–§12.4,背下修正口诀,手写 Treap、跳表与 k-d 树
2. **第二阶段(1 小时):**读 §12.5–§12.6,理解配对堆与 Python 选型惯例
3. **第三阶段(实验,1 小时):**完成章末实验:跳表字典的实现与对比
4. **验收标准:**能随口说出"红黑树高度 $O(\log n)$ " "跳表期望 $O(\log n)$ ",知道 Python 有序映射的三种替代

本章知识导图

- §12.1 红黑树:五条性质与插入修正
- §12.2 Treap:随机优先级的树堆
- §12.3 跳表:多层链表的概率平衡
- §12.4 k-d 树:多维空间切分与最近邻
- §12.5 配对堆:摊还 $O(1)$ 合并的实用堆
- §12.6 标准库选型指南
- §12.7 全书总结:十二章知识地图

§12.1 红黑树:五条性质与插入修正

12.1.1 五条性质与"黑高"

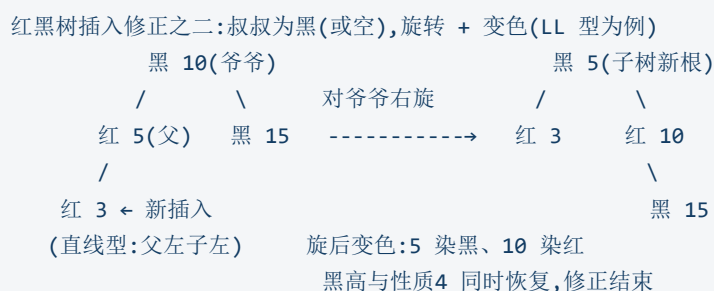
红黑树是一棵二叉查找树,每个结点带红/黑一色,满足五条性质:① 结点非红即黑;② 根是黑色;③ 叶结点(NIL)是黑色;④ 红结点的子结点都是黑色——"红结点不能有红孩子";⑤ 从任一结点到其所有后代叶的路径上,黑色结点数相同,这个数叫**黑高**。

关键保证:设黑高 h_b ,最短路径(全黑)长 h_b ,性质4 又禁红红相邻,最长路径至多 $2h_b$ ——最长 $\leq$ 最短的 2 倍,树高 $O(\log n)$,三操作最坏 $O(\log n)$ 。这就是"弱平衡":不追求 AVL 的高度差 ≤ 1 。

12.1.2 插入修正:变色与旋转

插入先染**红**——红结点不影响性质5(黑高不变),只可能破坏性质4(红父红子)。修正只看叔叔颜色:叔叔红 $\rightarrow$ 变色;叔叔黑 $\rightarrow$ 旋转。

示例 12-1 插入修正:叔叔为黑 $\rightarrow$ 旋转 + 变色



叔叔为红时,父、叔变黑,爷爷变红,冲突上移一层继续检查;爷爷若是根最后染黑。叔叔为黑时按 AVL 的四种形态旋转:LL/RR 直线一次旋转,LR/RL 之字形两次旋转;旋转后被提上来的结点染黑、爷爷染红。

★ 重点:插入易、删除难

插入只处理"红父红子"一处冲突,沿路径上移 $O(\log n)$ 层;删除要把黑高"挖掉"一层,处理"双黑"问题——红黑树"实现极复杂、接口极简单",删除从不手写,因此适合内建在标准库。

12.1.3 与 2-3-4 树的关系

一句话:**红黑树是 2-3-4 树的二叉表示**——把 2-3-4 树一个结点内的 2~3 个键"压扁"成红黑交替的二叉链。

12.1.4 Python 视角:没有 map,但有三种替代

C++ 有 `std::map` (红黑树),Java 有 `TreeMap`; Python 标准库没有平衡树——因为 `dict` 平均 $O(1)$ 已覆盖大多数场景,少数"要有序"的场景用三种替代:① 小数据 `list` + `bisect` 二分定位;② 批处理 `dict` + `sorted()`;③ 大数据上第三方 `sortedcontainers.SortedDict` (红黑树实现)。

C 的手写有序查找表	Python 的替代
手写 AVL/红黑树:数百行,修正分支众多	小数据: <code>list</code> + <code>bisect</code> ;大数据: <code>SortedDict</code>
有序遍历要自己写中序递归	<code>sorted(d.keys())</code> 一行,或 <code>SortedDict</code> 直接迭代
范围查询手写"第一个 $\geq x$ 的结点"	<code>bisect_left/right</code> 现成定位
忘记 <code>free</code> 整棵树即泄漏	垃圾回收自动回收全部结点

⚠ 误区 1:性质4 的方向与口诀记反

性质4 是"红结点的孩子必须黑",反过来"黑结点的孩子必须红"是错的——黑结点孩子颜色任意。高频错误还有口诀记反:叔叔红时去旋转,树永远修不好。

§12.2 Treap:随机优先级的树堆

12.2.1 思想:BST + 堆序

BST 退化根源是"输入顺序决定树形"。Treap(tree+heap)给每个结点一个**随机优先级**,并要求其满足**堆序**(小顶堆:父 $\leq$ 子)。键决定"在哪里",优先级决定"谁当根"——优先级随机,树形就随机,期望高度 $O(\log n)$ 。把平衡从"旋转纠错"换成"随机决定",实现从几百行降到几十行。

12.2.2 插入与旋转

插入:先按 BST 规则插到叶位置,再沿路径向上旋转,直到父的优先级 $\leq$ 子——与二叉堆"上滤"异曲同工。删除同理:把目标旋转着推下去,变叶子再摘除。

示例 12-5 Treap 完整实现(第一段:结构、旋转、插入)

```

# Treap:键满足 BST 序,随机优先级满足小顶堆堆序
import random

class Node:
    __slots__ = ("key", "pri", "left", "right") # 省内存
    def __init__(self, key, pri):
        self.key = key
        self.pri = pri
        self.left = self.right = None

def rotate_right(t):          # 右旋:提升左孩子
    x = t.left
    t.left = x.right
    x.right = t
    return x

def rotate_left(t):          # 左旋:提升右孩子
    x = t.right
    t.right = x.left
    x.left = t
    return x

def insert(t, key, pri):
    if t is None:
        return Node(key, pri)
    if key < t.key:
        t.left = insert(t.left, key, pri)
        if t.left.pri < t.pri:
            t = rotate_right(t)          # 堆序上滤
    elif key > t.key:
        t.right = insert(t.right, key, pri)
        if t.right.pri < t.pri:
            t = rotate_left(t)
    return t

```

| 示例 12-6 Treap 完整实现(第二段:删除与查找)

```
def erase(t, key):
    if t is None:
        return None
    if key < t.key:
        t.left = erase(t.left, key)
    elif key > t.key:
        t.right = erase(t.right, key)
    elif t.left is None:
        return t.right
    elif t.right is None:
        return t.left
    elif t.left.pri < t.right.pri:          # 优先级小的旋上来
        t = rotate_right(t)
        t.right = erase(t.right, key)
    else:
        t = rotate_left(t)
        t.left = erase(t.left, key)
    return t

def find(t, key):
    while t is not None and t.key != key:
        t = t.left if key < t.key else t.right
    return t is not None
```

12.2.3 期望 $O(\log n)$ 的直觉与缺陷

直觉:根是整棵树优先级最小的结点,优先级随机,所以"根是第 k 小的键"的概率均匀为 $1/n$,左右子树递归同构——与快排"随机枢轴"的分析同构:Treap 就是快排的树形版本。最坏情形(优先级恰好按键单调)概率 $1/n!$,现实不会出现。优先级相等时须约定比较方向,否则可能破坏 BST 序。

⚠ 误区 2:堆序方向与旋转方向配错

本例用小顶堆(父 $\leq$ 子),口诀"左小右旋、右小左旋";若定义成大顶堆,所有比较都要取反。实现后用有序序列插入再中序遍历:输出必须升序,且每个结点优先级 $\leq$ 子结点。

§12.3 跳表:多层链表的概率平衡

12.3.1 多层链表与几何分布

有序链表查找 $O(n)$,因为每步只能走一个结点。跳表(skip list)的思路:多建几层链表,上层是下层的"快速通道"。每个新结点先进入最底层,再以概率 $1/2$ 晋升,再以概率 $1/2$ 继续.....层数 k 满足几何分布: $P(\text{层} \geq k) = 2^{-(k-1)}$,每层结点数期望 $n/2^{k-1}$ 。查找从最高层出发,每层右移到"下一个更大"结点前再下降,共 $O(\log n)$ 层。

12.3.2 三操作:查找、插入、删除

三操作共享同一"下探"骨架:从最高层向右试探,把每层"最后一个 $< \text{key}$ 的结点"记进 `update` 列表。**查找**:下探后看 `update[0].next[0]` 是否为 `key`。**插入**:随机层数 `lvl`,若高于当前最高层先补 `update` 头指针,再逐层链入。**删除**:下探后若第 0 层后继即目标,逐层跳过,最后回收空高层。C 里每层指针要手写 `Node* next[k]`,Python 用 `next = [None] * (lvl + 1)` 一行搞定, `__slots__` 还能省内存。

示例 12-7 跳表完整实现(第一段:结构、查找、插入)

```

import random

class Node:
    __slots__ = ("key", "next")          # next[i]:第 i 层的后继
    def __init__(self, key, lvl):
        self.key = key
        self.next = [None] * (lvl + 1)  # 层号 0..lvl 均有效

class SkipList:
    def __init__(self, max_lvl=16):
        self.head = Node(0, max_lvl)    # 头结点:全部层的起点
        self.max_lvl = max_lvl
        self.level = 0
        self.rng = random.Random(12345) # 固定种子:可复现

    def random_level(self):               # 几何分布:每层 1/2 概率晋升
        lvl = 1
        while self.rng.getrandbits(1) and lvl < self.max_lvl:
            lvl += 1
        return lvl

    def find(self, key):
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
        x = cur.next[0]
        return x is not None and x.key == key

    def insert(self, key):
        if self.find(key):
            return                                # 字典语义:键唯一
        update = [None] * len(self.head.next)
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
            update[i] = cur                      # 每层最后一个 < key
        lvl = self.random_level()
        if lvl > self.level:
            for i in range(self.level + 1, lvl + 1):
                update[i] = self.head
            self.level = lvl
        x = Node(key, lvl)
        for i in range(lvl + 1):
            x.next[i] = update[i].next[i]
            update[i].next[i] = x

```

| 示例 12-8 跳表完整实现(第二段:删除与按序遍历)

```

def erase(self, key):
    update = [None] * len(self.head.next)
    cur = self.head
    for i in range(self.level, -1, -1):
        while cur.next[i] and cur.next[i].key < key:
            cur = cur.next[i]
        update[i] = cur
    x = cur.next[0]
    if x is None or x.key != key:
        return False
    for i in range(self.level + 1):
        if update[i].next[i] != x:    # 该层没有 x
            break
        update[i].next[i] = x.next[i]
    while self.level > 0 and self.head.next[self.level] is None:
        self.level -= 1                # 回收空高层
    return True

def keys(self):
    v = []
    p = self.head.next[0]
    while p:
        v.append(p.key)
        p = p.next[0]
    return v

def __len__(self):
    return len(self.keys())

```

12.3.3 与平衡树对比:期望 vs 确定

对比项	跳表	红黑树 / AVL
平衡机制 / 实现难度	随机层数,~ 链表级,几十行	旋转 + 不变式,高,数百行
高度	期望 $O(\log n)$,最坏 $O(n)$	最坏保证 $O(\log n)$
每结点内存	平均约 2 个指针	2 个指针 + 颜色位
工程使用	Redis 有序集合 zset、LevelDB	std::map/set、Java TreeMap

确定性与实现难度不可兼得:要最坏保证(实时系统)选红黑树;要快速实现选跳表——跳表还天然便于无锁并发修改。

C 的手写多级链表	Python 的跳表类
每层一个 <code>Node* next[k]</code> 数组要手写分配	<code>next = [None] * (lvl + 1)</code> 一行
释放多层结点容易重复 free 或泄漏	垃圾回收自动回收, <code>__slots__</code> 省内存
随机层数用 <code>rand()</code> ,可移植性差	<code>random.Random(种子)</code> 质量与可复现性都有保证

⚠ 误区 3:随机层数用错了分布

层数必须**几何分布**(每层 1/2 概率继续)。三个常见错误:① 固定层数让高层退化成线性扫描 $O(n)$;② 删除后不回收空高层;③ 用 `time()` 做种子,同一秒插入的结点层数高度相关。调试:逐层打印链表,验证"第 0 层完整、高层稀疏"。

§12.4 k-d 树:多维空间切分与最近邻

12.4.1 交替维度切分建树

k-d 树递归地选一个维度取**中位数**作切分点,左子树为小于切分点的点集,右子树反之;下一层换另一个维度。对二维点:第 0 层按 x 、第 1 层按 y 、第 2 层再按 x **维度交替,保证每个区域在两个方向都被"削"过**,树高 $O(\log n)$ 。C++ 用 `nth_element` 线性选中位数;Python 直接用 `sorted` 或 `statistics.median` 选,代价同为 $O(n \log n)$ 建树。

示例 12-9 k-d 树:交替维度建树(二维)

```
def build(pts, lo, hi, depth):
    """pts 就地重排后返回子树根;返回 (根, 新列表) 也行,这里用下标切分"""
    if lo > hi:
        return None
    dim = depth % 2                                # 维度交替:0 按 x,1 按 y
    mid = (lo + hi) // 2
    pts[lo:hi + 1] = sorted(pts[lo:hi + 1], # 中位数切分
                             key=lambda p: p[dim])
    node = {"p": pts[mid], "dim": dim, "left": None, "right": None}
    node["left"] = build(pts, lo, mid - 1, depth + 1)
    node["right"] = build(pts, mid + 1, hi, depth + 1)
    return node
```

12.4.2 最近邻查询与剪枝

给定查询点 q ,找点集中距离最近者。先沿树下探到 q 所在区域,得到"当前最佳距离" d_{best} ;回溯时——若 q 到分割线的距离 $\geq d_{\text{best}}$,另一侧任何点都不可能更近,整棵子树剪掉。

示例 12-10 k-d 树:最近邻查询(带剪枝)


```
def dist2(a, b):
    dx, dy = a[0] - b[0], a[1] - b[1]
    return dx * dx + dy * dy

def nearest(node, q):
    """返回 (最佳距离平方, 最佳点)"""
    best_d, best_p = float("inf"), None
    def dfs(node):
        nonlocal best_d, best_p
        if node is None:
            return
        d = dist2(node["p"], q)
        if d < best_d:
            best_d, best_p = d, node["p"]
        dim = node["dim"]
        cut = node["p"][dim]
        qc = q[dim]
        near, far = (node["left"], node["right"]) if qc < cut \
            else (node["right"], node["left"]) # 先探近侧
        dfs(near)
        d_cut = (qc - cut) * (qc - cut) # q 到分割线距离
        if d_cut < best_d: # 剪枝:够近才跨过去
            dfs(far)
    dfs(node)
    return best_d, best_p
```

记号说明:距离平方省去开根号

最近邻只需比较大小, `dist2` 返回距离平方即可,省掉每步一次 `sqrt`;剪枝条件用平方比较数学等价。

12.4.3 应用与退化警告

k-d 树是"多维点集查询"的默认答案:游戏引擎的"最近敌人"查询替代逐对检测的 $O(n^2)$,光线追踪与 k-NN 分类也建立在"空间切分 + 剪枝"上。数据挤在一条直线上时剪枝失效,查询最坏 $O(n)$;生产级替代是 R 树或网格索引。

误区 4:剪枝条件漏写或写反

最近邻两种经典错误:① 漏掉 `if d_cut < best_d` 直接递归两侧——正确但退化成全树搜索 $O(n)$;② 比较方向反了——可能跳过真正更近的点。调试:拿 5 点小数据集逐结点打印"近侧/远侧/是否剪枝"核对。

§12.5 配对堆:摊还 $O(1)$ 合并的实用堆

12.5.1 配对堆是什么

第6章的二叉堆支持 `insert` 与 `deleteMin`,但**合并两个堆**要 $O(n)$ 。左式堆 `merge` 是 $O(\log n)$ 但实现复杂;**配对堆** (pairing heap)走实用路线: `merge` 只比两个根的键,把小根挂到大根的孩子链表里,摊还 $O(1)$; `deleteMin` 把根的孩子两两配对合并,摊还 $O(\log n)$ 。

示例 12-11 配对堆:`merge`、`insert`、`deleteMin`

```

# 配对堆:孩子用"左孩子 + 右兄弟"链表表示,根最小
class PHNode:
    __slots__ = ("val", "child", "next")
    def __init__(self, val):
        self.val = val
        self.child = None      # 第一个孩子,next 为兄弟链
        self.next = None

def merge(a, b):              # 摊还 O(1)
    if a is None:
        return b
    if b is None:
        return a
    if a.val > b.val:
        a, b = b, a          # 键小的做根
    b.next = a.child
    a.child = b
    return a

def merge_pairs(first):       # 两趟合并:摊还 O(log n)
    if first is None or first.next is None:
        return first
    a, b = first, first.next
    rest = b.next
    return merge(merge(a, b), merge_pairs(rest))

def delete_min(root):
    children = root.child
    return merge_pairs(children)  # 根即最小,直接摘掉

# insert = merge(现有堆, 单结点堆),即摊还 O(1)

```

12.5.2 三堆选择的一句话

只做优先队列(不合并):二叉堆;需要合并且代码量敏感:配对堆;需要最坏保证的合并:左式堆。代价是摊还而非最坏:单次 deleteMin 可能偶发 $O(n)$,长时间平均 $O(\log n)$ 。Python 工程上优先队列直接用 `heapq` 即可;配对堆的价值在于理解"摊还"与"可合并堆"这两个概念。

§12.6 标准库选型指南

12.6.1 Python 没有 map:三种替代的决策表

C++ 同学随手写 `std::map`,Python 的标准答案随数据规模走: `list` + `bisect`、`dict` + `sorted`、第三方 `sortedcontainers`。怎么选:

需求	<code>list</code> + <code>bisect</code>	<code>dict</code> + <code>sorted</code>	<code>SortedDict(sortedcontainers)</code>
按键升序遍历	列表本身有序	不支持(需 <code>sorted</code>)	支持(中序遍历)
范围查询 <code>[lo, hi]</code>	支持(<code>bisect</code> 定位)	不支持	支持(<code>irange</code>)
查找 / 插入	$O(\log n)$ / $O(n)$	平均 $O(1)$ / 平均 $O(1)$	$O(\log n)$
适用规模	小($\leq 10^4$)	任意(无序)	大且持续有序

12.6.2 bisect:有序列表的二分工具箱

`bisect_left(a, x)` 返回"第一个 $\geq x$ "的下标, `bisect_right` 返回"第一个 $> x$ "的下标——配合 `list.insert` 就是一棵"数组版平衡树"的插入;配合切片就是范围查询:

示例 12-12 用 bisect 做有序字典(小数据)

```
from bisect import bisect_left, bisect_right

keys, vals = [], []
def d_insert(k, v):
    i = bisect_left(keys, k)
    if i < len(keys) and keys[i] == k:
        vals[i] = v
    else:
        keys.insert(i, k)
        vals.insert(i, v)

def d_find(k):
    i = bisect_left(keys, k)
    return vals[i] if i < len(keys) and keys[i] == k else None

lo, hi = bisect_left(keys, 4), bisect_right(keys, 8)
print(keys[lo:hi])           # 范围查询 [4, 8]
```

12.6.3 手写 vs 标准库:有序字典并排

把 §12.3 的跳表包一层,接口与 `SortedDict` 对齐——第3章 ADT 的威力:

示例 12-13 手写跳表字典 vs SortedDict,接口并排

```
# ---- 手写版:§12.3 的 SkipList 包成有序字典 ----
class SkipDict:
    def __init__(self):
        self._sl = SkipList()
    def insert(self, k): self._sl.insert(k)
    def find(self, k):   return self._sl.find(k)
    def erase(self, k): return self._sl.erase(k)
    def keys(self):     return self._sl.keys()
    def __len__(self):  return len(self._sl)

d = SkipDict()
d.insert(5); d.insert(3)
print(d.keys())           # [3, 5]

# ---- 标准库版:同一套接口,红黑树实现(第三方库) ----
from sortedcontainers import SortedDict
m = SortedDict({3: "three", 5: "five", 8: "eight"})
print(list(m))            # [3, 5, 8]
print(m.irange(4, 8))     # 范围查询 [4, 8]
```

12.6.4 工程选型建议

- 一次性有序输出:任意容器 + `sorted()` (C++ 要手写排序或 `std::sort`)
- 持续插入 + 需要有序:小数据 `list` + `bisect`,大数据 `SortedDict`

- 只查不遍历: dict (平均 $O(1)$);注意 dict 自 3.7 起保持插入顺序,但**不是**按键序
- 顺序敏感又不愿装库:dict + 用时 `sorted(d.keys())` ,适合批处理

C 的做法	Python 的做法
<code>qsort(a, n, sz, cmp)</code> 传函数指针	<code>sorted(a, key=...)</code> 传 key 函数或 lambda
map 底层红黑树,排序是容器特性	dict 无序;要序用 <code>bisect/SortedDict</code>
比较器写错静默产生错误排序	<code>sorted(key=)</code> 越简单越不易错,还可 <code>functools.cmp_to_key</code> 迁移 C 比较器

§12.7 全书总结:十二章知识地图

12.7.1 十二章知识地图

- 第1章 绪论:递归与数学 → 第2章 算法分析: $O/\Omega/\Theta$
- 第3章 表栈队列:数组与链表 → 第4章 树:BST、AVL、遍历
- 第5章 散列:冲突处理 → 第6章 堆:二叉堆、上滤下滤
- 第7章 排序:八大排序 → 第8章 不相交集:union/find
- 第9章 图:最短路与 MST → 第10章 算法设计:贪心、分治、DP
- 第11章 摊还分析:三方法 → 第12章 高级结构:平衡与期望

12.7.2 "遇到问题选什么结构"终极决策表

需求场景	推荐结构	理由
只做等值查找/去重	散列(dict / set)	平均 $O(1)$,常数最小
需要按键有序遍历	跳表或 SortedDict	底层链/中序即有序序列
范围查询 [lo, hi]	bisect 定位有序列表	$O(\log n)$ 定位 + $O(k)$ 输出
优先队列(含合并)	heapq / 配对堆	$O(\log n)$ / 合并摊还 $O(1)$
多维点集最近邻	k-d 树	平均 $O(\log n)$ 剪枝查询

12.7.3 下一步学习方向

学完十二章,你已具备"手写任何基础结构 + 正确选型"的能力,三个方向任选:① **算法竞赛**——跳表与 Treap 常作为平衡树的快速替代;② **系统设计**——缓存用散列、排行榜用跳表、范围索引用 B+ 树;③ **数据库索引**——B+ 树是红黑树的"磁盘亲戚"。最后记住:数据结构没有银弹,选型的本质是拿时间复杂度、实现复杂度与最坏保证做三角权衡。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
查找 / 插入 / 删除	红黑树(SortedDict 底层)	$O(\log n)$	$O(\log n)$
有序遍历全部元素	红黑树中序 / SortedDict 迭代	$O(n)$	$O(n)$
前驱 / 后继	有序列表 bisect 定位	$O(\log n)$	$O(\log n)$
查找 / 插入 / 删除	Treap	期望 $O(\log n)$	$O(n)$ (优先级恰好有序)
查找 / 插入 / 删除	跳表	期望 $O(\log n)$	$O(n)$ (全部同层)
建树	k-d 树(sorted 切分)	$O(n \log n)$	$O(n \log n)$
最近邻查询	k-d 树	平均 $O(\log n)$	$O(n)$ (数据退化)
合并两个堆	配对堆	摊还 $O(1)$	摊还 $O(1)$
插入(单结点)	配对堆	摊还 $O(1)$	摊还 $O(1)$
deleteMin	配对堆	摊还 $O(\log n)$	摊还 $O(\log n)$
按键查找	dict(散列)	平均 $O(1)$	$O(n)$ (散列冲突)
有序遍历	sorted(d.keys())	$O(n \log n)$	$O(n \log n)$
范围查询 [lo, hi]	bisect 定位有序列表	$O(\log n) + O(k)$	同左(k 为输出个数)
二分查找	bisect_left / bisect_right	$O(\log n)$	$O(\log n)$
插入到有序列表	list.insert(bisect 定位后)	$O(n)$	$O(n)$

最小必会清单

- 能默写红黑树五条性质,并解释"性质4+5 $\rightarrow$ 高度 $O(\log n)$ "
- 能画出插入修正两个分支(叔叔红变色、叔叔黑旋转)与四种旋转形态
- 能说出"红黑树是 2-3-4 树的二叉表示"
- 能独立实现 Treap 与跳表的插入/删除/查找,说出期望为何 $O(\log n)$
- 能说出跳表与红黑树的取舍(期望 vs 确定、实现难度、内存、并发)
- 能独立实现 k-d 树交替维度建树与最近邻剪枝查询
- 能说出配对堆合并摊还 $O(1)$ 、deleteMin 摊还 $O(\log n)$,一句话对比三堆
- 能说出 Python 有序映射三替代(list+bisect、dict+sorted、SortedDict)及各自复杂度

自测题

Q 1. 为什么性质4 与性质5 一起能保证红黑树高度 $O(\log n)$?

✓ 参考答案

性质5 说所有路径黑高相同,设黑高 h_b ;性质4 禁红红相邻,最长路径至多 $2h_b$,最短路径 h_b ——最长 $\leq 2 \times$ 最短。而黑高 h_b 的树至少 2^{h_b-1} 个内部结点,故树高 $\leq 2 \log_2(n+1) = O(\log n)$ 。

Q 2. Treap 的期望高度为什么是 $O(\log n)$? 最坏情形是什么?

✓ 参考答案

优先级随机时,根是"优先级最小结点"的概率均匀为 $1/n$,左右子树递归同构——与快排随机枢轴同构。最坏是优先级恰好按键单调,高度 $O(n)$,概率 $1/n!$ 。

Q 3. 跳表查找从第几层开始?层数为什么用几何分布而不是固定层数?

✓ 参考答案

从当前最高层开始逐层下降。几何分布(每层 $1/2$ 概率晋升)使每层期望结点数 $n/2^k$,查找每层只走常数步,期望 $O(\log n)$;固定层数则退化成线性扫描。

Q 4. k-d 树最近邻查询的剪枝条件是什么?漏写会怎样?

✓ 参考答案

回溯时,若查询点到分割线的距离平方 $\geq$ 当前最佳距离平方,另一侧不可能更近,整棵子树剪掉;只有严格小于才进入。漏写则退化成全树搜索 $O(n)$;方向写反则可能漏掉更近的点。

Q 5. Python 里要"按键有序的映射",从小到大数据量各有什么方案?各是什么复杂度?

✓ 参考答案

小数据($\leq 10^4$): `list` + `bisect`,查找 $O(\log n)$ 、插入 $O(n)$;批处理: `dict` + 用时 `sorted(d.keys())`,整体 $O(n \log n)$;大数据且持续有序:第三方 `SortedDict` (红黑树),查找/插入 $O(\log n)$ 。标准库没有平衡树,这是与 `C++ std::map` 最大的差异。

章末实验题:跳表字典的实现与对比

实验 12:跳表字典的实现与对比

实验目标:实现完整跳表并封装成有序字典,与 `dict` 对照验证正确性,实测 10^4 级随机键耗时。

实验要求:

- 任务 1:实现跳表的查找/插入/删除与随机层数(知识点:\$12.3 三操作与几何分布)
- 任务 2:封装成有序字典接口 `insert/find/erase/keys`(知识点:\$12.3 底层链有序 + 第3章 ADT)
- 任务 3:用 `dict` 做同一组随机操作,验证输出一致(知识点:\$12.6 标准库对照)
- 任务 4:对 10^4 级随机键分别计时插入与查找(知识点:\$12.3 期望复杂度 + 第2章计时)
- 任务 5:删除全部元素,验证跳表为空(知识点:\$12.3 删除与空层回收)

实验环境:Python 3.8+。运行: `python lab12.py`。

第一步 示例 12-7/12-8 实现 `SkipList`,升/降/乱序三组输入冒烟测试,`keys()` 必须升序

第二步 任务 3 用同一种子生成随机操作序列,插入前先 `find` 去重;任务 4 洗牌生成 10^4 个不重复键,结果累加输出防死代码消除

✅ 参考答案(完整可运行程序,约 100 行,分四段)

```
# lab12.py — 跳表字典的实现与对比(第一段:任务1 跳表核心)
import random
import time

class Node:
    __slots__ = ("key", "next")
    def __init__(self, key, lvl):
        self.key = key
        self.next = [None] * (lvl + 1)    # 层号 0..lvl 均有效

class Skiplist:
    # 任务 1:三操作 + 随机层数
    def __init__(self, max_lvl=16):
        self.head = Node(0, max_lvl)
        self.max_lvl = max_lvl
        self.level = 0
        self.rng = random.Random(12345)    # 固定种子:可复现

    def random_level(self):
        # 几何分布:每层 1/2 概率晋升
        lvl = 1
        while self.rng.getrandbits(1) and lvl < self.max_lvl:
            lvl += 1
        return lvl

    def find(self, key):
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
        x = cur.next[0]
        return x is not None and x.key == key

    def insert(self, key):
        if self.find(key):
            return    # 字典语义:键唯一
        update = [None] * len(self.head.next)
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
            update[i] = cur
        lvl = self.random_level()
        if lvl > self.level:
            for i in range(self.level + 1, lvl + 1):
                update[i] = self.head
            self.level = lvl
        x = Node(key, lvl)
        for i in range(lvl + 1):
            x.next[i] = update[i].next[i]
            update[i].next[i] = x
```

lab12.py(第二段:删除、遍历 + 任务2 字典封装)

```

def erase(self, key):
    update = [None] * len(self.head.next)
    cur = self.head
    for i in range(self.level, -1, -1):
        while cur.next[i] and cur.next[i].key < key:
            cur = cur.next[i]
        update[i] = cur
    x = cur.next[0]
    if x is None or x.key != key:
        return False
    for i in range(self.level + 1):
        if update[i].next[i] != x:
            break
        update[i].next[i] = x.next[i]
    while self.level > 0 and self.head.next[self.level] is None:
        self.level -= 1          # 回收空高层
    return True

def keys(self):
    v = []
    p = self.head.next[0]
    while p:
        v.append(p.key)
        p = p.next[0]
    return v

def __len__(self):
    return len(self.keys())

class SkipDict:
    # 任务 2:有序字典接口
    def __init__(self):
        self._sl = SkipList()
    def insert(self, k): self._sl.insert(k)
    def find(self, k):   return self._sl.find(k)
    def erase(self, k): return self._sl.erase(k)
    def keys(self):     return self._sl.keys()
    def __len__(self):  return len(self._sl)

```

lab12.py(第三段:任务3 一致性对照 + 任务4 计时)


```

def main():
    # 任务 3:与 dict 对照,输出必须一致
    print("== 任务3:跳表字典 vs dict 一致性 ==")
    d = SkipDict()
    ref = {}
    rng = random.Random(12345)
    for i in range(20000):
        k = rng.randrange(100000)
        op = i % 4                # 0/1 插,2 删,3 查
        if op <= 1:
            if k not in ref:
                d.insert(k)
            ref[k] = k
        elif op == 2:
            d.erase(k)
            ref.pop(k, None)
        else:
            if d.find(k) != (k in ref):
                print("不一致! key=", k)
                return 1
    dk, mk = d.keys(), sorted(ref)
    print("20000 次随机操作后:跳表 len=%d,dict len=%d" % (len(d), len(ref)))
    print("跳表前 10 键:", dk[:10])
    print("按键序与 sorted(dict) 一致:", dk == mk)

    # 任务 4:10^4 随机键计时
    N = 10000
    ks = list(range(N))
    rng.shuffle(ks)                # 洗牌为随机序列
    s2 = SkipDict()
    t0 = time.perf_counter()
    for k in ks:
        s2.insert(k)
    sl_ins = (time.perf_counter() - t0) * 1000
    ref2 = {}
    t0 = time.perf_counter()
    for k in ks:
        ref2[k] = k
    d_ins = (time.perf_counter() - t0) * 1000
    hits = 0
    t0 = time.perf_counter()
    for k in ks:
        hits += 1 if s2.find(k) else 0
    sl_find = (time.perf_counter() - t0) * 1000
    t0 = time.perf_counter()
    for k in ks:
        hits += 1 if k in ref2 else 0
    d_find = (time.perf_counter() - t0) * 1000
    print("\n== 任务4:10^4 随机键计时 ==")
    print("插入 %d 键:跳表 %.4f ms,dict %.4f ms" % (N, sl_ins, d_ins))
    print("查找 %d 键:跳表 %.4f ms,dict %.4f ms" % (N, sl_find, d_find))
    print("(命中数 %d,防止死代码消除)" % hits)

```

lab12.py(第四段:任务5 删除全部 + 入口)

```
# 任务 5:删除全部元素,验证跳表为空
for k in ks:
    s2.erase(k)
print("\n== 任务5:删除全部后 ==")
print("跳表 len=%d,为空=%s,find(0)=%s"
      % (len(s2), len(s2) == 0, s2.find(0)))

if __name__ == "__main__":
    main()
```

示例输出(真实运行,机器不同数值不同,量级关系一致):

```
== 任务3:跳表字典 vs dict 一致性 ==
20000 次随机操作后:跳表 len=9281,dict len=9281
跳表前 10 键: [1, 14, 21, 26, 31, 36, 38, 46, 47, 48]
按键序与 sorted(dict) 一致: True

== 任务4:10^4 随机键计时 ==
插入 10000 键:跳表 27.6136 ms,dict 0.4609 ms
查找 10000 键:跳表 14.3060 ms,dict 0.3026 ms
(命中数 20000,防止死代码消除)

== 任务5:删除全部后 ==
跳表 len=0,为空=True,find(0)=False
```

设计说明:① update 列表统一"下探路径",插入/删除共享骨架;② 插入前先 find 去重,与 dict 键唯一语义对齐;③ 任务3 用同一随机操作序列同时驱动两者,任何一次查找不一致立即退出;④ 同一份键序列计时,跳表与 dict 同为 $O(\log n)$ 对 $O(1)$,dict 常数更小;删除后回收空高层,len=0 验证删除路径正确。对照标准库用 dict + sorted ,正是 §12.6 的 Python 惯例。

全书十二章到此结束:第1-2章打下递归与复杂度地基,第3-9章建起线性结构、树、散列、堆、排序、图的工具箱,第10-11章学会设计范式与摊还分析,本章补上最后一块拼图——平衡与概率。数据结构是算法工程师的母语,愿你在竞赛、系统设计或工程实践中带着这十二章的判断力走得更远。